Module Breakdown & Build Plan

ECITY — Mobile Shop Management Web App

Version 1.6 · 2026

15 modules, sequenced for one developer

Companion documents: PRD, Module Breakdown, Engineering Design, Deployment Guide, Database Guide

1. How to Read This Document

The application is split into **14 modules (M0–M13)**, ordered so that each one is buildable, testable and demonstrable on its own, and only depends on modules already finished. Build them in order. Do not start a module until its dependencies are marked done.

Each module lists:

- **Goal** — what the business can do once it ships
- **Delivers** — feature IDs from the PRD (which map to the source Feature List v3 section numbers)
- **Data model** — the tables introduced or extended
- **Screens** and **Server work** — the concrete surface to build
- **Done when** — acceptance criteria; if you cannot demonstrate all of them, the module is not done
- **Depends on** and **Effort** — a full-time-equivalent estimate for one experienced developer

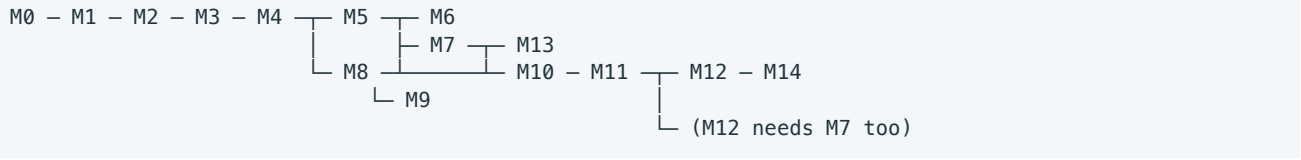
> **Estimates assume one developer working full time**, including tests and review. At roughly 20 productive hours a week, multiply by about two for a calendar estimate.

2. Delivery Map

#	Module	Effort (FTE weeks)	Depends on	Release
M0	Foundations: project, auth, RBAC, branch context, audit	2.0	—	Alpha
M1	Master data: business, branches, users, customers, suppliers	1.5	M0	Alpha
M2	Inventory core: products, branch stock, device units (IMEI)	2.5	M1	Alpha
M3	Purchases & supplier ledger	2.0	M2	Alpha
M4	Sales & billing & invoices	3.0	M3	Alpha
M5	Payments, credit sales & customer dues	1.5	M4	Alpha
M6	Returns, exchange & trade-in	2.0	M5	Beta
M7	Money: expenses, cash drawer, bank accounts, daily closing	2.5	M5	Beta
M8	Branch transfers & stock adjustments	2.0	M4	Beta
M9	Global search & IMEI device history	2.0	M8	Beta
M10	Dashboards & analytics	3.0	M7, M8	v1.0
M11	Reports, exports, imports & opening balances	2.0	M10	v1.0
M12	**External billing system integration (Excel feed)**	**3.5**	M7, M11	v1.0
M13	Notifications, alerts & warranty tracking	1.5	M7	v1.0
M14	Backup, data protection, hardening & go-live	1.5	all	v1.0
	Total	**~32.5 FTE weeks**		

Add roughly 15% for discovery, rework and the open questions in PRD §11 — plan for **36–38 weeks full time**, or about **8–9 months at 20 hours a week**.

2.1 Dependency shape



M6, M7 and M8 are independent of each other once M5 is done — if a second developer joins, that is the point to split.

2.2 Rules that apply to every module

1. **Branch scope and permission are enforced on the server** in every endpoint you add. Never rely on the UI.
2. **Money is integer paise.** No floats, ever.
3. **Stock, money and device-status changes are written in the same database transaction as their document.**
4. **Nothing is hard-deleted.** Documents move to Cancelled / Voided / Reversed states.
5. **Every device-touching action appends a `device_event` row.** M9 is only possible because M2–M8 did this faithfully.
6. **State goes in the right tier.** Server data (inventory, sales, customers) is read by Server Components or TanStack Query — never copied into a client store. Shared browser-only state uses Zustand, and the only thing that qualifies before M10 is the bill being built (M4). Everything else is `useState`. M0–M3 need no store at all.
7. Every module ends with: migrations committed, seed/demo data updated, tests green, and a five-minute demo of the "Done when" list.

M0 — Foundations

Goal. A running, deployed, secured skeleton: someone can log in, land in a branch context, and every action they take is audited.

Delivers. FR-1.1 – FR-1.5, FR-31.1, plus the technical base for everything else.

Data model. `business`, `user`, `role`, `permission`, `role_permission`, `user_branch`, `session`, `audit_log`.

Screens. Login, forgot/reset password, app shell (sidebar, header, branch switcher, user menu), user list, user create/edit with role and branch assignment, role editor, audit log viewer with filters.

Server work.

- Session-based authentication, password hashing, password reset tokens with expiry.
- A single authorisation helper used by every endpoint: `requirePermission(user, permission, branchId)`.
- Request-scoped branch context: the selected branch (or "all branches" for those permitted) resolved once and applied to every query.
- Audit writer invoked from a common data-access layer so that new features are audited by default rather than by remembering.
- Database migrations, seeding, error handling, structured logging, health check.
- CI: typecheck, lint, tests, build an image, push to GHCR, deploy over SSH. Provision the Hetzner server, Docker Compose stack (app, Postgres 16, pg-boss worker, Caddy) and the staging stack — follow the Deployment Guide.

Done when.

- A user can log in, reset a password and log out; sessions expire and can be revoked.
- An Admin can create a Manager restricted to Branch A, and that Manager cannot read Branch B data — verified by calling the API directly, not just by the hidden menu.
- Every create/update/delete writes an audit row with user, timestamp, entity and old/new values.
- ~~The app is deployed to staging from the main branch automatically, and the production stack serves HTTPS on the real domain.~~ **Deferred — see below.**

> **Deferred from M0: the staging deployment.** The CI and deploy workflows are written and the code is on GitHub, but no server has been provisioned. A server costs about ₹550/month and buys nothing until there is something worth showing the shop owner. **Provision it at the end of Alpha (after M5)**, when one branch can transact end to end and the owner can try it on real hardware. Until then, migrations are tested locally. This is a deliberate deferral, not a skipped criterion: M0 is otherwise complete, and this line moves to M5's "Done when".

Depends on. — **Effort.** 2.0 weeks.

M1 — Master Data

Goal. The shop is configured and the people it trades with exist.

Delivers. FR-2.1 – FR-2.4, FR-3.1, FR-3.2, FR-3.8, FR-5.10, FR-6.6, FR-10.1.

Data model. `branch`, `tax_rate`, `payment_method`, `expense_category`, `customer`, `supplier`, `attachment`.

Screens. Business profile & logo, tax configuration, payment methods, branch list / create / edit / deactivate, customer list + profile + create/edit, supplier list + profile + create/edit.

Server work. CRUD with validation; duplicate detection on customer phone and supplier phone/GST; deactivation semantics (a deactivated branch or supplier is hidden from new-transaction pickers but remains fully readable in history); attachment upload with signed URLs.

Done when.

- Business profile, tax rates and payment methods drive the pickers used later.
- Branches can be created, edited and deactivated; a deactivated branch cannot be chosen for a new transaction but its history is intact.
- Customer and supplier profiles exist with all PRD fields, are searchable by name/phone/email, and show an (empty for now) history tab.

Depends on. M0. **Effort.** 1.5 weeks.

M2 — Inventory Core

Goal. Stock exists, in branches, and every handset is an individually tracked device with an IMEI, a main type and a status. **This is the module that decides whether the rest of the system is correct** — take the extra day here.

Delivers. FR-4.1 – FR-4.12, FR-5.1 – FR-5.7 (the classification, identifier and status rules).

Data model.

- `category`, `brand`, `product` (accessory or mobile model), `branch_stock` (product × branch: qty, min_qty)
- `device_unit`: product_id, variant, ram, storage, colour, purchase_price, selling_price, tax, warranty, supplier_id, purchase_date, **main_type** (`NEW|USED|ER|ACT|GLOBAL`), **is_new_cut** + new-cut details,

current_branch_id, status, `primary_imei` (cached for display only)

- `sales_channel` on `device_unit`: `ECITY` | `EXTERNAL` | `BOTH`, defaulted from `main_type` (NEW → `EXTERNAL`) — needed by M12 so a device billed in the other system can never be sold here. Add it now; retrofitting it after M4 means revisiting the billing screen
- `source` on `device_unit`, `purchase` and `sale`: `ECITY` | `LEGACY` — where the record came from. One column, added now, saves a migration later
- `device_identifier`: device_id, **value**, **type** (`IMEI` | `SERIAL`), slot, is_primary — **a device has one or more identifiers**. Never model these as `imei_1` / `imei_2` columns on the device; the whole point of the separate table is that a third identifier, or a laptop serial, costs nothing later
- `category.identifierType`: `IMEI` | `SERIAL` | `NONE` — what a serialised category's units are identified by
- `battery_health_percent` on `device_unit`: nullable 1–100, checked in the database. Blank for sealed new stock; it drives used and trade-in pricing
- `device_event` (append-only): device_id, seq, event_type, occurred_at, branch_id, from_branch_id, to_branch_id, ref_type, ref_id, actor_id, payload
- `stock_ledger` (append-only) for non-serialised quantity movement

Constraints to enforce in the database, not only in code.

- `is_new_cut = true` is permitted only when `main_type = 'GLOBAL'` (check constraint).
- `device_identifier.imei` unique across the entire business.
- Exactly one `is_primary` identifier per device (partial unique index).
- `sales_channel = 'EXTERNAL'` blocks the device from ECITY sale (enforced in M4).
- `branch_stock.qty >= 0`.

Screens. Product list with search and kind/inactive filters; product create/edit, with image upload on the edit page once the product exists; device list with filters for branch, main type, GLOBAL/NEW CUT, brand, category, supplier and status; device detail page (a static view now — M9 turns it into the full timeline); low-stock view.

Manual device entry. A small create-device form is also needed, for opening stock and corrections — this module's own acceptance criteria describe form behaviour (`imei_slots` showing one field or two), which nothing else provides until M3. It is deliberately secondary: labelled *Add manually*, and it states that supplier stock belongs in a purchase. The bulk path is M3's IMEI-capture grid.

Not phones only. The five main types and NEW CUT apply to every serialised item — laptops, MacBooks, speakers, tablets. What differs is the identifier: `category.identifierType` is `IMEI` for phones and `SERIAL` for other electronics, and it drives both the database format check and the label on the form. Accessories stay counted, with no identifier.

Server work. Stock read/write helpers that every later module calls (`increaseStock`, `decreaseStock`, `setDeviceStatus`), each of which writes the ledger/event row automatically. Get this API right once and M3–M8 become simple.

Device creation and update take `imeis: string[]`, never a pair of named fields, and every lookup resolves a device by *any* of its identifiers. Add the `imei_slots` business setting (default **1**) — it governs **how many input fields the UI renders and nothing else**. The API, importer, search and reports always accept and return the full list, so raising the setting later is a configuration change with no migration, no backfill and no code release.

Done when.

- A device can be created with each of the five main types, and a NEW CUT device can only be created under GLOBAL — rejected at the API and by the database for the other four.
- A device created through the API with three IMEIs stores all three, marks one primary, and is found by searching any of them; the same IMEI on a second device is rejected with a message naming the first.

- With `imei_slots = 1` the form shows exactly one IMEI field, and setting it to 2 in **Settings** → **Business** shows two with no deployment.
- A laptop or speaker registers against a serial number rather than an IMEI, carries the same five main types, and is refused a letter-bearing value where an IMEI is expected.
- A product's stock reads as a **number** in both cases: counted products sum their branch stock, serialised ones count the units still in stock. Two identical handsets show as 2.
- Low stock lists counted products at or below their branch minimum.
- The device list filters correctly by every filter in FR-4.6, including *normal GLOBAL* vs *GLOBAL + NEW CUT*.
- Accessory stock is per branch; the same product shows different quantities in two branches.
- Every status change writes a `device_event` row; no code path changes a device without one.

Depends on. M1. **Effort.** 2.5 weeks.

On PRD OQ-1 and OQ-2: neither blocks the start of this module. The five main types are stored as codes; what **ER** and **ACT** stand for is a display label in configuration, not a schema concern. Ask during M2 whether either carries a sub-designation the way GLOBAL carries NEW CUT — if one does, it is a nullable column plus a check constraint, the same shape as `is_new_cut`.

M3 — Purchases & Supplier Ledger

Goal. Stock legitimately enters a branch, and what is owed to suppliers is tracked.

Delivers. FR-5.8 – FR-5.15, FR-14.1 – FR-14.3.

Data model. `purchase`, `purchase_item`, `supplier_payment`, `supplier_ledger_entry`, purchase attachments.

Screens. Purchase list with filters; purchase entry form (supplier, branch, date, items) with a fast IMEI-capture grid for mobile lines — scan IMEI, pick main type, mark NEW CUT when GLOBAL. The grid renders `imei_slots` IMEI columns, which is **one column in v1**, while the request body it submits is always a list; purchase detail with attachments; supplier payment entry; supplier outstanding report; supplier profile history tab now populated.

Server work.

- Confirming a purchase, in one transaction: create device units for each IMEI, increase accessory stock, write `device_event: purchased/received`, post the supplier ledger entry, set payment status.
- Reversal: a purchase can be reversed only if none of its devices have moved on. If any have, the error must name the blocking IMEIs.
- Purchases record their `source` (`ECITY` by default), so stock bought through the other system can later be imported without ambiguity.
- Duplicate IMEI detection at entry time, checked against **every** identifier of every device in the business, with a clear message naming the conflicting device.

Done when.

- Confirming a purchase with 5 mobiles and 20 accessories raises branch stock correctly and registers 5 device units with the right main types, each with its IMEI list stored and a primary identifier set.
- Partially paying a purchase leaves the correct supplier outstanding, and the supplier profile shows the purchase, the payment and the balance.
- Reversing an untouched purchase removes the stock and the ledger effect and leaves an audit trail; reversing a purchase whose device was sold is refused with a specific reason.

Depends on. M2. **Effort.** 2.0 weeks.

M4 — Sales & Billing

Goal. The counter can sell, and produce an invoice. The highest-traffic screen in the product.

Delivers. FR-6.1 – FR-6.8, FR-26.1, FR-26.3, FR-26.4, FR-26.5, FR-38.2, FR-38.3.

Data model. `sale`, `sale_item`, `sale_payment`.

Built as: no separate `invoice_series` table was needed. M3 already introduced `document_sequence`, which allocates gapless per-branch, per-year counters under `SELECT … FOR UPDATE`; invoices reuse it with a different document kind. One table, one concurrency proof, one place to audit.

Screens. The billing screen — a single keyboard-driven page: search by name/SKU/barcode/IMEI, cart with line discounts and tax, customer attach/create inline, split payment across methods, save & print. Sale list with filters. Sale detail. Printable invoice in A4 and 80 mm thermal layouts, plus PDF download and share. Customer history (FR-6.7) on the customer record: spend, purchases across every branch, and what is still owed.

Built as: the customer, supplier and product fields are **searchable pickers**, not `<select>`s. A capped dropdown silently omits records — found in production-shaped data at 571 suppliers and 693 serialised products against a 500 cap — and the omission is invisible to the user.

Client state — the first module that needs a store. The bill being built is real client state: cart lines, per-line and bill-level discounts, the attached customer, split payments across methods, and later the trade-in from M6. Four or five sibling components read and write it — the search box, the cart, the totals panel, the payment panel — which is past what props or context handle cleanly.

- Use **Zustand** (`npm i zustand`), one store scoped to the billing screen. Not Redux: same result, a quarter of the code.
- **Persist the store to** `localStorage`, including the bill's idempotency key. This is what satisfies PRD §9.3 — a dropped connection or an accidental refresh must not lose a half-built bill, and re-submitting must not create a second one.
- Clear the store only on a confirmed save, never optimistically.
- **TanStack Query** joins here too, for client-side reads (product and IMEI search as the user types). Keep the division strict: Query owns anything Postgres owns; Zustand owns only what exists in the browser. Never copy sale or stock data into the store.

Built as: Zustand with `persist` as planned. TanStack Query was **deferred** — the search box is the only client-side read in M4, and a debounced `fetch` covers it without adding a second data layer. Revisit when a screen needs shared caching or background refetch.

Server work.

- Availability check and device lock at save: the exact device must be `In Stock` at the selling branch, and a conditional update prevents two tills selling the same IMEI.
- **Channel check:** a device with `sales_channel = 'EXTERNAL'` is refused at search time and at save time, with the message *"This device is billed through the other system."* This is what makes M12 safe — it removes the possibility of the same phone being invoiced in both systems.
- **Mark as sold externally**, a permissioned action setting `SOLD_PENDING_IMPORT`: stock drops now, and M12's upload completes the record with the real invoice. This is the fallback for `BOTH`-channel devices.
- Tax computation honouring inclusive/exclusive configuration.

Built as: **OQ-4 was answered "yes — statutory".** So the tax engine also splits every line into CGST/SGST (intra-state) or IGST (inter-state), and the invoice carries HSN/SAC per line, the place of supply, and an HSN-wise

summary. Place of supply follows the customer's registered state and falls back to the branch's; where neither is known the sale stays intra-state rather than guessing IGST. The split is stored, not derived at print time, because a reprint years later must be identical even if the branch or customer has since moved state. CGST takes the floor of the halving and SGST the remainder, so the two always add back to exactly the tax charged.

Built as: **OQ-7 was answered "no — a reliable connection is acceptable".** The persisted cart still covers a refresh or a dropped connection, and idempotency still prevents a double bill, but saving requires the server. Offline-first would be its own module.

- Invoice number allocation, gapless per branch series, safe under concurrency.
- Idempotent sale submission keyed by a client-generated id, so a retried request after a dropped connection does not create a second bill.
- Sale completion in one transaction: stock down, `device_event: sold`, payments posted, invoice numbered.

Done when.

- An accessory-only cash sale completes in under 20 seconds using only the keyboard and scanner.
- A mobile sale shows the main type on the line, and NEW CUT detail for a GLOBAL device.
- Selling an IMEI that another user has just sold fails clearly rather than double-selling; the device's status is `Sold` exactly once.
- A NEW device cannot be added to a bill at all, and the refusal explains why.
- Both invoice formats print correctly, and the branch invoice series has no gaps or duplicates after 200 concurrent test sales.
- A half-built bill survives a browser refresh and a dropped connection, and submitting it twice creates exactly one sale.

Depends on. M3. **Effort.** 3.0 weeks. *OQ-4 and OQ-7 both answered during M4 — see below.*

M5 — Payments, Credit Sales & Customer Dues

Goal. Credit is controlled: what is owed, by whom, since when, and where it was collected.

Delivers. FR-7.1 – FR-7.6.

Data model. `customer_payment`, `customer_payment_allocation`, `customer_ledger_entry`, credit fields on `sale` (due_date, credit_notes).

Built as: there is deliberately **no stored** `outstanding` **column**. It is always summed from the append-only ledger (docs/03 §4.2), so the screen and the books cannot disagree. The customer side mirrors the supplier side table for table, which means one mental model and one place to fix allocation or voiding. `customer_ledger_entry` is append-only, enforced by a database trigger rather than by convention.

Screens. Payment collection against one or many open sales; customer dues list with aging; customer statement; overdue view; payment receipt print.

Server work. Ledger-based balance, never a stored mutable number: outstanding is always derived and reconcilable. Allocation of a payment across invoices. Automatic status flip to Paid when settled. Both the sale branch and the collection branch recorded on the payment.

Built as:

- **One definition of "what a sale has been paid".** Counter payments (`sale_payment`) and later collections (`customer_payment_allocation`) are summed by a single shared SQL expression used by the sale detail, the sale list, the payment-status filter and the customer history. Before M5 there were four separate copies; leaving

them would have meant a bill settled by a receipt still showing UNPAID in the list.

- **Receipts are gaplessly numbered** through the same `document_sequence` machinery as invoices, per-branch where the branch has its own prefix.
- **A voided receipt settles nothing** — the allocations stay as a record of what the receipt claimed, and the ledger gets a REVERSAL entry rather than an edit. The invoice reopens automatically.
- `customer_payment.void` **is its own permission.** Counter staff may collect (a customer clearing their tab is the counter's job) but may not erase a collection already recorded.
- **Allocation runs inside the caller's transaction.** The first cut queried the pool from inside an open transaction and deadlocked under concurrency — 25 parallel receipts hung for 70 seconds. It also did two queries per open invoice; it is now one.
- **The agreed terms appear on the bill.** FR-7.2 says a credit sale stores its due date and notes; storing them without showing them would mean the salesperson agreed a date nobody could see again. An unpaid sale carries an outstanding figure, the due date (in red once past), the note, and a link straight to collection.
- **Branch-wise (FR-7.6) reports two different figures deliberately.** A branch is shown what it is *owed* (on bills it raised) beside what it *collected* (money taken at its counter, whoever raised the bill). They are not meant to match: a customer may buy at one shop and settle at another, and a branch doing the group's collecting should look like it. Collections are bounded by a date range, defaulting to the current month, because "collected" with no period is not a number anyone can act on.

Done when.

- A credit sale creates the correct outstanding and due date; three partial payments settle it and flip it to Paid.
- A payment taken at Branch B against a sale made at Branch A records both branches and appears in both branches' cash/collection figures correctly.
- Customer outstanding recomputed from the ledger equals the displayed balance for every customer in the test data.
- Aging buckets 0–7 / 8–30 / 31–60 / 60+ are correct against hand-checked dates.
- **Carried from M0:** the Hetzner server is provisioned, and pushing to `main` deploys to staging automatically. Alpha is the first release someone outside the project sees, so it needs somewhere to live. See the Deployment Guide.

Built as: the pipeline is written, and the deployment guide §6.2 has the half that needs a provider account. **The provider changed:** Hetzner's Singapore region never sold the CX22 this plan assumed (the CX line is EU-only), and their prices rose 144–190% on 15 June 2026, which made them both the dearest option and the furthest away. The recommendation is now an India region — Vultr Mumbai or DigitalOcean Bangalore — which is cheaper, ~20 ms instead of ~150 ms, and keeps the shop's records in India. Nothing in the pipeline was provider-specific, so no code changed; see deployment guide §1.1. Fixing the workflow found three faults that would each have broken a real deploy: it did not wait for CI (`needs: []`), the migration step ran `drizzle-kit` from an image that does not contain it with `|| true` swallowing the failure, and the compose file referenced a `worker.js` that does not exist. It also now runs `db:sync-roles`, without which a module that adds a permission ships a screen nobody can open.

Depends on. M4. **Effort.** 1.5 weeks.

M6 — Returns, Exchange & Trade-In

Goal. Goods come back and old phones come in, without corrupting stock, money or device history.

Delivers. FR-8.1 – FR-8.5, FR-9.1 – FR-9.3.

Data model. `sales_return`, `return_item`, `refund`, `trade_in`, device inspection fields.

Screens. Return by invoice / customer / IMEI; full, partial and exchange return flows; inspection queue for returned devices with the classify action (Available / Used / Damaged / Repair Required); trade-in capture inside the billing screen, with valuation and difference payable — extends M4's Zustand cart store rather than introducing a second one.

Server work.

- Returned mobiles go to `Returned / Inspection`, never straight back to sellable (this is the rule most likely to be got wrong).
- Refund posts against a payment method and the branch cash drawer or account.
- Trade-in devices are created as new device units in the receiving branch with the correct main type and GLOBAL/NEW CUT information, and their own `device_event` chain begins.
- Main type and NEW CUT survive the whole return path unchanged.

Done when.

- A returned GLOBAL + NEW CUT device still reads as GLOBAL + NEW CUT after return and reclassification.
- A returned device is not sellable until an authorised user classifies it Available.
- An exchange sale records the trade-in value, the difference paid, the new device sold and the old device received, all linked to one another.

Depends on. M5. **Effort.** 2.0 weeks.

M7 — Expenses, Cash Drawer, Bank Accounts & Daily Closing

Goal. The day balances. Expected money is compared with counted money, per branch, every day.

Delivers. FR-10.1 – FR-10.3, FR-11.1 – FR-11.5, FR-12.1 – FR-12.4, FR-13.1 – FR-13.5.

Data model. `expense`, `cash_drawer_day`, `cash_movement`, `account`, `account_transaction`, `daily_closing`.

Screens. Expense entry and list with receipt upload; cash drawer day view showing opening, every cash in/out, expected cash; accounts list with balances, receipts, payments, transfers and reconciliation; daily closing screen with the day summary, expected vs actual per method, computed difference, and confirm-close; closing history.

Server work.

- Every cash-affecting event from M4–M6 (cash sales, credit collections, refunds, expenses, supplier payments) posts a `cash_movement` into the branch's open drawer day. Backfill this into the earlier modules as part of this module's work.
- `Expected Cash = Opening + Cash In − Cash Out`, derived from movements, not stored.
- Closing a day freezes it; editing anything dated inside a closed day requires an explicit permission and is audited.
- **Legacy sales take cash into the same physical drawer.** Expected cash is therefore wrong until M12's daily file is imported. Build the closing screen so it can be gated on "today's external feed imported", with an authorised override that records a reason. Wire the gate itself in M12; leave the hook here.
- Opening balance of the next day carries from the previous day's actual count.

Done when.

- A day with cash sales, a UPI sale, a credit collection, a refund, an expense and a supplier payment produces the correct expected cash, and entering a counted amount ₹200 short shows a ₹200 shortage attributed to that branch, user and timestamp.

- Two branches close independently and the consolidated reconciliation report adds up.
- A staff user cannot alter a transaction inside a closed day; an Admin can, and it is audited.

Depends on. M5. **Effort.** 2.5 weeks. *Blocked on PRD OQ-5.*

M8 — Branch Transfers & Stock Adjustments

Goal. Stock moves between branches under control, and physical/system differences are corrected on the record instead of silently.

Delivers. FR-3.6, FR-3.7, FR-28.1 – FR-28.3.

Data model. `stock_transfer`, `transfer_item`, `stock_adjustment`.

Screens. Transfer request (choose source, destination, devices by IMEI and/or accessory quantities); approval queue; dispatch (mark In Transit); receiving screen at the destination that scans IMEIs and flags anything missing or unexpected; transfer history; stock adjustment entry with reason code and evidence.

Server work.

- The state machine **Requested** → **Approved** → **In Transit** → **Received**, with Cancelled available up to receipt, enforced server-side; illegal transitions rejected.
- In-transit devices belong to neither branch's sellable stock — they must not appear as available anywhere.
- Receipt moves the exact IMEI to the destination branch and writes `device_event: transferred_out` / `transferred_in` with both branch ids and the date.
- Adjustments record branch, product or IMEI, main type, GLOBAL/NEW CUT, reason, user and timestamp, and require permission.

Done when.

- A device transferred A → B cannot be sold at A once dispatched, cannot be sold at B until received, and after receipt carries a movement history showing both branches with dates.
- A cancelled in-transit transfer returns the devices to the source branch cleanly.
- A miscount adjustment changes accessory stock and appears in the audit log and in the inventory movement report.

Depends on. M4. **Effort.** 2.0 weeks.

M9 — Global Search & IMEI Device History

Goal. The flagship feature: one search box, everywhere, and an IMEI that opens a device's whole life.

Delivers. FR-30.1 – FR-30.7.

Data model. No new business tables — this module *reads* `device_event` and the documents it points to. Add full-text search indexes and a materialised search view over products, customers, suppliers, invoices, SKUs, barcodes and IMEIs.

Screens. A command-palette style global search available on every screen (keyboard shortcut), with results grouped by type — Devices, Products, Customers, Suppliers, Invoices — and a dedicated **Device History** page: identity header, commercial summary, current position, and a vertical timeline `Purchase → Seller → Branch → Transfers → Sale → Customer → Return/Repair/Other`, each entry

linking to its source document.

Server work.

- One search endpoint that detects the input shape (15-digit IMEI, phone number, email, invoice number, free text) and routes accordingly, always filtered by the user's branch permissions. IMEI matching runs against `device_identifier`, so **any** of a device's identifiers finds it.
- Device history assembly: one query over `device_event` ordered by sequence, joined to purchases, transfers, sales, returns and adjustments.
- Indexes to hit the < 500 ms and < 1.5 s targets in PRD §9.1.

Done when.

- Searching a full IMEI opens the device page directly; searching a partial IMEI lists candidates; searching the second or third IMEI of a multi-IMEI device opens the same page as its primary one.
- For a device that was purchased, transferred twice, sold on credit, returned and reclassified, the timeline shows all seven stages with dates and working links, and the identity header shows main type, NEW CUT status and every IMEI on the device with the primary one marked.
- A Branch-A-only user searching a Branch B customer's phone number gets no results, verified at the API.
- Device history for a device with 50 events renders in under 1.5 s.

Depends on. M8 (so that every event type exists to display). **Effort.** 2.0 weeks.

M10 — Dashboards & Analytics

Goal. The numbers. Branch dashboards, the owner's consolidated view, and the nine analytics areas.

Delivers. FR-15.1 – FR-15.3, FR-16 – FR-24, FR-35.1 – FR-35.3, FR-36.1 – FR-36.4.

Data model. Reporting views / summary tables. Consider nightly-refreshed rollups per branch per day (sales, cost, profit, units, payment mix) so that year-range queries stay fast; today's figures read live.

Screens. Branch dashboard; owner consolidated dashboard with branch comparison; and analytics pages for sales, product, brand, customer, credit, inventory, profit, payment and supplier — each with date range, branch selector, comparison mode, chart plus table, and drill-through.

Server work.

- One analytics query layer with a consistent shape: date range, branch set, grouping, comparison period.
- The mobile-type dimension is mandatory across product, inventory, profit and insights analytics: five main types reported separately, GLOBAL split into normal vs NEW CUT.
- Every analytics query carries a `source` **filter** (ECITY / LEGACY / both, defaulting to both) so the owner can see the whole business, or just what each system did. Add the dimension now; M12 supplies the LEGACY rows.
- Inventory movement `Opening → Purchases → Sales → Returns → Adjustments → Current` reconstructed from the stock ledger and device events.
- Drill-down routing Business → Branch → Transaction → Product/IMEI, terminating in the M9 device history page.

Done when.

- Every measure listed in PRD §6.15 exists and matches a hand-calculated result on the seeded test dataset.
- Switching the branch selector between one branch, two branches and all branches changes every figure correctly.

- Product and inventory analytics can show GLOBAL, and within GLOBAL split NEW CUT out separately.
- A 12-month analytics range for 10 branches returns in under 5 seconds.
- Clicking a dashboard number reaches an individual IMEI in at most four clicks.

Depends on. M7, M8. **Effort.** 3.0 weeks.

M11 — Reports, Exports, Imports & Opening Balances

Goal. Data gets in at the start and out whenever it is needed.

Delivers. FR-25.1 – FR-25.4, FR-33.1 – FR-33.3, FR-34.1 – FR-34.3.

Data model. `import_job`, `import_row_error`, `export_job`.

Screens. Report centre (sales, purchase, inventory, financial, credit, tax, reconciliation, branch comparison) with saved filters; export buttons producing CSV, Excel and PDF; import wizard — upload, map columns, validate, preview, commit — with a downloadable per-row error report; opening balance screens for inventory, cash/accounts and existing customer/supplier dues.

Server work. Streaming exports so large files do not exhaust memory; import validation that rejects a whole file on structural errors but reports row-level errors individually; imports carry branch, main type, GLOBAL/NEW CUT and **multiple IMEI columns per device row** (accepted even while `imei_slots` is 1), and run through the same service layer as manual entry so that stock, ledger and device events stay consistent.

Done when.

- All seven report families produce correct output for a branch and consolidated, and export to CSV, Excel and PDF.
- Inventory reports group the five main types with the GLOBAL/NEW CUT breakdown.
- Importing 1,000 devices with 20 deliberate errors imports 980, reports the 20 with row numbers and reasons, and leaves no partial rows; rows carrying two IMEIs import both.
- Opening balances produce correct starting stock, cash and customer/supplier outstanding, visible in the dashboards.

Depends on. M10. **Effort.** 2.0 weeks. *Blocked on PRD OQ-9.*

M12 — External Billing System Integration (Excel Feed)

Goal. The shop keeps using its existing billing system for NEW items. This module makes that survivable: their Excel export becomes a controlled daily feed into ECITY, stock stays truthful, and the same IMEI can never be sold twice.

Delivers. FR-38.1 – FR-38.14.

The problem this solves

Two systems touch the same physical stock. The legacy system bills NEW items; ECITY bills everything else and owns inventory, credit, cash and reporting. Without a design, three things break:

1. **Double sale.** The legacy system sells IMEI X at 11 AM. ECITY still shows it In Stock. At 4 PM someone sells it again in ECITY. Two invoices, one phone.

2. **Cash never tallies.** Legacy sales take real cash into the same physical drawer. ECITY's expected cash is short by exactly that amount until the file is uploaded, so daily closing is meaningless.
3. **Reports lie.** Sales, profit and stock figures are missing everything the other system did.

The design: block by channel, do not race the clock

The instinct is to mark stock as sold and let the nightly upload confirm it. That works, but it still leaves a window in which ECITY believes a device is sellable. **Close the window instead of shrinking it.**

Every device carries a `sales_channel`: `ECITY`, `EXTERNAL`, or `BOTH`, defaulted from `main_type` (NEW → `EXTERNAL`, everything else → `ECITY`) and overridable per device.

- The ECITY billing screen **refuses** to sell a device whose channel is `EXTERNAL`, with a plain message: *"This device is billed through the other system."*
- So ECITY's stock is never *dangerously* wrong. It may be a few hours stale — it still shows the device as held — but nobody can double-sell it.
- The daily upload converts those devices to `Sold`, attaching the real invoice number, date, customer and price.

For anything sold in ECITY that the legacy system also handles (`BOTH`), a manual action **Mark as sold externally** sets the status `SOLD_PENDING_IMPORT`. Stock drops immediately, and the upload later completes the record with the real invoice. This is the fallback path, not the main one.

The Excel format is unknown — build everything except the last 100 lines

The file layout cannot be designed for yet. **Structure the code so that only one small file needs writing when it arrives:**

```
/server/services/external-import/
  types.ts              <- canonical row shapes. STABLE. Write this first.
  adapters/
    legacy-sales.adapter.ts      <-- TODO: the ONLY file that knows the
    legacy-purchase.adapter.ts   <-- real Excel column layout
  mapping/
    legacy.mapping.json <- column-name -> canonical-field map, editable
                           without a code change or redeploy
  ingest.ts             <- upload, parse, stage, validate, dedupe
  match.ts              <- resolve IMEI/product/customer to our records
  apply.ts              <- commit via the SAME services as manual entry
  reconcile.ts          <- daily comparison + exception report
```

Everything downstream of `types.ts` is written against the canonical shape and can be built **now**, before anyone has seen the file:

```
// types.ts - the contract. Does not change when the Excel changes.
type ExternalSaleRow = {
  externalInvoiceNo: string
  externalLineId:    string        // for idempotency
  soldAt:            Date
  imei?:             string        // mobiles
  sku?:              string        // accessories
  qty:               number
  unitPrice:         bigint        // paise
  discount:          bigint
  tax:               bigint
  paymentMethod?:    string
  customerName?:     string
  customerPhone?:    string
  branchCode?:       string
  raw:               Record<string, unknown>   // the untouched source row
}
```

When the real file arrives, the work is: read the columns, fill in `legacy.mapping.json`, write ~100 lines in the adapter, add fixture tests from a real export. **Two to three days**, not a redesign. Prefer the JSON mapping over

hard-coded column names — the legacy vendor will change a header eventually, and that should be a config edit, not a release.

Data model. `import_batch` (file, hash, uploaded_by, period, status, counts), `import_row` (batch, row number, raw payload, canonical payload, match result, applied ref, error), `external_reference` (our entity ↔ their invoice/line id, unique), and on existing tables: `source` (`ECITY` | `LEGACY`) plus `sales_channel` and `SOLD_PENDING_IMPORT` on `device_unit`.

Screens. Upload page with drag-and-drop and batch history. The wizard's step state is server-backed (`import_batch` / `import_row`), so it needs no client store — a browser crash mid-review must not lose a staged batch; **preview before commit** showing what will be created, matched, skipped and rejected, with per-row reasons; exception queue for unmatched rows with resolve actions (link to an existing device, create it, ignore with a reason); daily reconciliation report; a "today's feed not yet uploaded" banner on the dashboard.

Server work.

- **Idempotency.** Key every row on `(source, externalInvoiceNo, externalLineId)` and store the file hash. Re-uploading the same file changes nothing; re-uploading a corrected file updates only what actually differs.
- **Two-phase: stage then apply.** Parse and validate into `import_row` first, show the operator the preview, and only commit on confirm. Nothing is half-applied — the whole batch commits or none of it does.
- **Apply through the existing service layer.** Imported sales call the same `createSale()` / `sellDevice()` functions as the counter, so stock, ledgers, `device_event` rows and analytics stay consistent by construction. Never write directly to tables from the importer.
- **Unmatched rows.** A sale of an unknown IMEI is *not* auto-created — it goes to the exception queue, because selling something we never bought is a data gap worth a human look. Purchase rows for unknown IMEIs *do* auto-create the device with `source = LEGACY`.
- **Reversal.** A whole batch can be reversed, producing reversal documents rather than deletions.
- **Daily closing gate (M7).** A branch's day cannot be closed until today's legacy file is uploaded, or an authorised user records an explicit override with a reason. Otherwise expected cash is guaranteed to be wrong and the mismatch figure is noise.

Done when.

- Uploading the same file twice produces zero duplicate sales.
- A NEW device cannot be sold on the ECITY billing screen; the message explains why.
- A file with 500 rows, of which 20 are unmatched IMEIs and 5 malformed, previews accurately, commits the 475, and queues the 25 with reasons.
- After committing a sales file, stock, customer dues, cash figures, profit and the IMEI device history all reflect the legacy sales identically to how they would if typed in by hand.
- The device history of a legacy-sold phone shows the sale with its real invoice number and is labelled as coming from the other system.
- A branch cannot close its day with today's file missing, unless overridden with a reason that appears in the audit log.
- Reversing a batch cleanly restores stock and balances.

Depends on. M7 (cash and closing) and M11 (shares the import infrastructure). **Effort.** 3.5 weeks — of which **3.0 is format-independent and can start immediately**, and 0.5 is the adapter once a real export is in hand. *Blocked on PRD OQ-10 and OQ-11.*

M13 — Notifications, Alerts & Warranty

Goal. The system tells you what needs attention instead of waiting to be asked.

Delivers. FR-27.1 – FR-27.3, FR-29.1, FR-29.2.

Data model. `notification`, `notification_rule`, warranty fields on `device_unit`.

Screens. Notification centre with unread state and per-user preferences; alert cards on the dashboards; warranty view on device history; warranty expiry list.

Server work. A scheduled job evaluating the rules — low stock, overdue customer payments, supplier dues, cash mismatch, unclosed day, stock adjustment, warranty expiry — writing notifications scoped to branch users, with consolidated versions for owners/admins. Deduplicate so the same condition does not notify daily forever.

Done when.

- Each of the seven triggers fires on a constructed test condition and reaches exactly the right users.
- A branch user sees only their branch's alerts; the owner sees all, grouped by branch.
- Warranty details appear on the device history page and an expiring-soon list is available.

Depends on. M7. **Effort.** 1.5 weeks. *Blocked on PRD OQ-6 if external channels are wanted.*

M14 — Backup, Data Protection, Hardening & Go-Live

Goal. The system is safe to trust with a real business.

Delivers. FR-31.2, FR-31.3, FR-32.1 – FR-32.3, and PRD §9 non-functional requirements.

Work.

- Three backup layers per the Deployment Guide §7: Hetzner snapshots, `pg_dump` to Cloudflare R2 via restic (7 daily / 4 weekly / 6 monthly), and a **restore actually performed** into a scratch database, timed, with the procedure written down.
- **Decide the acceptable data-loss window.** Nightly dumps alone mean losing up to a full trading day. Two ways to close that: dump every four hours (one crontab line, ~4-hour worst case), or add WAL archiving with `pgBackRest`/`wal-g` to R2 for true point-in-time recovery (~5-minute worst case, about half a day of setup). **Recommended: both** — this is money data.
- Owner-triggered full business data export.
- Soft-delete and reversal states audited across all financial and inventory documents; confirm no destructive path remains.
- Security pass: authorisation tests on every endpoint for role × branch, rate limiting on login and search, signed URLs on all uploads, dependency audit, secret handling.
- Performance pass against the PRD §9.1 targets on a dataset the size of the §9.1 sizing assumption; add the indexes the traces demand.
- Go-live: production environment, monitoring and error tracking, uptime alerting, opening balances loaded, staff training, and a two-week parallel-run period before paper records are retired. Staff training must cover the **daily external-feed upload** — it is a new habit the shop has to form, and everything downstream of it (stock, cash, closing, reports) is wrong on any day it is skipped.

Done when.

- A restore from backup into a clean environment is demonstrated and timed, and the chosen data-loss window is met in that test.
- An authorisation test suite covers every endpoint for each role and branch combination and passes.
- Performance targets are met on full-size data.
- Branch one has run live in parallel for two weeks with no unexplained discrepancy in cash, stock or dues.

Depends on. All. **Effort.** 1.5 weeks.

3. Suggested First Two Weeks

1. Answer PRD OQ-1, OQ-2, OQ-3, OQ-7, OQ-10 and OQ-11 with the shop owner, and **get one real Excel export from the existing billing system** — a single sample file unblocks 0.5 weeks of M12 and may change assumptions elsewhere — these four change the schema or the sales module, and are cheap to answer now and expensive to answer later.
2. Set up the repository, database, deployment and CI (start of M0).
3. Write the full schema for M0–M2 up front, even though you build it in stages, and include `device_identifier` as a separate table from the very first migration even though the UI will show a single IMEI field. The device, identifier and event tables are load-bearing for every later module, and rewriting them after M4 is the single most expensive mistake available in this project.